fn make_fully_adaptive_composition_queryable

Michael Shoemate

This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of `fn make_fully_adaptive_composition_queryable`.

1 Hoare Triple

Precondition

Compiler-verified

- Argument `input_domain` of type `DI`.
- Argument `input_metric` of type `MI`.
- Argument `output_measure` of type `MO`.
- Argument `data` of type `DI_Carrier`.
- Generic `DI` implements `Domain`.
- Generic `MI` implements `Metric`.
- Generic `MO` implements `Measure`.
- `(DI, MI)` implements `MetricSpace`.

User-verified

`data` is a member of `input_domain`.

Pseudocode

```
def make_sequential_odometer_queryable(
    input_domain: DI,
    input_metric: MI,
    output_measure: MO,
    data: DI_Carrier
) -> OdometerQueryable[MI, MO, Measurement[DI, TO, MI, MO], TO]:
    child_maps = []  # Vec<PrivacyMap<MI, MO>>
    def transition(  #
        self_: OdometerQueryable[MI, MO, Measurement[DI, TO, MI, MO], TO],
        query: Query[OdometerQuery[Measurement[DI, TO, MI, MO], MI_Distance]]
     ):
        # this queryable and wrapped children communicate via an AskPermission query
        # defined here, where no-one else can access the type
        @dataclass
        class AskPermission:
```

```
            id: usize

        match query:
            # evaluate external invoke query
            case Query.External(OdometerQuery.Invoke(measurement), query_wrapper):  #
                assert_components_match(  #
                    DomainMismatch,
                    input_domain,
                    measurement.input_domain
                )

                assert_components_match(  #
                    MetricMismatch,
                    input_metric,
                    measurement.input_metric
                )

                assert_components_match(  #
                    MeasureMismatch,
                    output_measure,
                    measurement.output_measure
                )

                if not output_measure.concurrent:
                    # when the output measure doesn't allow concurrent composition,
                    # wrap any interactive queryables spawned.
                    # This way, when the child gets a query it sends an AskPermission query to this parent queryable,
                    # giving this sequential odometer queryable
                    # a chance to deny the child permission to execute
                    child_id = child_maps.len()

                    seq_wrapper = Wrapper.new_recursive_pre_hook(  #
                        lambda: self_.eval_internal(AskPermission(child_id))
                    )
                else:
                    seq_wrapper = None

                wrapper = compose_wrappers(query_wrapper, seq_wrapper)  #

                answer = measurement.invoke_wrap(data, wrapper)  #

                # we've now increased our privacy spend. This is our only state modification
                child_maps.push(measurement.privacy_map)  #

                return Answer.External(OdometerAnswer.Invoke(answer))

            # evaluate external map query
            case Query.External(OdometerQuery.Map(d_in), _):
                d_out = output_measure.compose([pmap.eval(d_in) for pmap in child_map])
                return Answer.External(OdometerAnswer.Map(d_out))

            case Query.Internal(query):
                # check if the query is from a child queryable who is asking for permission to execute
                if isinstance(query, AskPermission):  #
                    # deny permission if the sequential odometer has moved on
                    if query.id + 1 != child_maps.len():
                        raise ValueError("sequential odometer has received a new query")

                    # otherwise, return Ok to approve the change
                    return Answer.internal(())


```

```
                raise ValueError("query not recognized")

    return Queryable.new(transition)  #
```

Postconditions

Theorem 1.1. For every setting of the input parameters `(input_domain, input_metric, output_measure, DI, TO, MI, MO)` to `make_fully_adaptive_composition_queryable` such that the given preconditions hold, `make_fully_adaptive_composition_queryable` raises an exception (at compile time or run time) or returns a valid measurement. A valid measurement has the following property:

1. (Privacy guarantee). For every pair of elements x, x' in `input_domain` and for every pair (`d_in`, `d_out`), where `d_in` has the associated type for `input_metric` and `d_out` has the associated type for `output_measure`, if x, x' are `d_in`-close under `input_metric`, `privacy_map`(`d_in`) does not raise an exception, and `privacy_map`(`d_in`) $\leq$ `d_out`, then `function`(x), `function`(x') are `d_out`-close under `output_measure`.

Proof. Wrapping Guarantee The query wrapper is labeled `query_wrapper` on line 20. `wrapper` on line 53 includes `query_wrapper` by the postcondition on `compose_wrappers`.

`Measurement.invoke_wrap` on line 55 guarantees to wrap all spawned IM queryables with the provided wrapper, satisfying the wrapping guarantee. Since this is the only location where interactive mechanism queryables can be spawned, the wrapping guarantee holds. □

Proof. Privacy guarantee The transition function follows the $\mathcal{G}$-$OdomCon(IM)$ procedure described in Algorithm 6 of [HST+23], with some adjustments that do not materially affect the privacy analysis.

- When an odometer invokes $\mathcal{M}_{k+1}$, the OpenDP Library implementation eagerly accounts for all future privacy loss of the child mechanism immediately (on line 58), even if the analyst never interacts with the spawned interactive mechanism.

- The OpenDP Library represents interactive mechanisms via queryables, where the state s is hidden from the adversary. When invoking $\mathcal{M}_k$, no initial state λ is passed, and the response is a queryable with hidden state. By the definition of a valid interactive measurement, the view an adversary gains from this queryable has privacy loss d_k for some choice of `d_in`.

- The queryable for child k can be viewed as $\mathcal{M}_k$ together with s_k. Since the queryable itself satisfies d_k-DP, the queryable can be returned to the user to provide query access, child queryables ($\mathcal{M}_k$ and s_k) can be removed from the odometer state, and the handling of child update queries $m = (j, q)$ can be removed while preserving equivalency with Algorithm 6. This equivalently reduces the odometer state to $(s, [d_1, ...d_k])$.

- Instead of storing the privacy losses d_k for some `d_in`in the state, a vector of functions is stored that can compute d_k when given `d_in`. The choice of `d_in`is left implicit in the paper, in regards to $\mathcal{M}_k$ satisfying $d - \mathcal{D}$-DP; whereas `d_in`is explicit and delayed in the implementation. The equivalent odometer state is now $(s, [\text{map}_1(\cdot), ...\text{map}_k(\cdot)])$.

- In addition to storing the dataset x, the state also contains the input domain, input metric and privacy measure. Each incoming mechanism must share these same supporting elements. This is an implicit requirement of the paper made explicit in the implementation.

We now discuss how the pseudocode implements this equivalent algorithm. The input domain, input metric, privacy measure and data are moved into the transition function, as well as a mutable list of privacy loss maps from line 7, to form the state.

First consider the case where the privacy measure satisfies concurrent composition. Lines 27 and 33 are necessary to guarantee that the constructed odometer queryable gives valid privacy loss guarantees with

respect to `input_metric` and `output_measure`. On line 55, the user-verified preconditions for invoking the measurement are met, by the precondition that the `data` is a member of `input_domain` and the check that the measurement's input domain matches `input_domain` on line 21. Therefore by the definition of a valid measurement, `answer` satisfies `measurement.privacy_map(d_in)`-DP, with respect to `output_measure`, for some choice of `d_in`.

Now consider the case where the privacy measure does not satisfy concurrent composition. Then `seq_wrapper` on line 47 is a wrapper that will cause wrapped queryables to send an `AskPermission` query with their self-reported id to this queryable before answering any query. By line 69, permission is only granted if the most recently spawned child is asking for permission to execute a query. This ensures that an adversary may only interact with the most recently spawned child mechanism.

Now that we've shown a correspondence between the pseudocode and Algorithm 6 of [HST+23], the proof from Appendix B.1 shows that the pseudocode implements a valid $\mathcal{D}$ DP privacy loss accumulator for interactive mechanisms.

TODO: connect this formalization with the definition of BasicCompositionMeasure.

The transition function is then used to construct a queryable on line 80. □

References

[HST+23] Samuel Haney, Michael Shoemate, Grace Tian, Salil Vadhan, Andrew Vyrros, Vicki Xu, and Wanrong Zhang. Concurrent composition for interactive differential privacy with adaptive privacy-loss parameters, 2023.